

Chapter 41: Two Pointers in Depth

Personal Notes

You already met two pointers in your Array Practice sets — reversing an array, checking a palindrome, printing extremes alternately. Those were the entry-level uses. In this chapter we go deeper: 3-Sum, Container With Most Water, fast-slow pointer for cycle detection, and a suite of variants that show up in half the medium-hard array/string interviews at Google.

The unifying idea is small: instead of nested loops examining pairs, USE TWO INDICES that move intelligently through the data. Each pointer moves forward at most n times, so the total work stays $O(n)$ even when you're conceptually visiting $O(n^2)$ pairs. Once you internalize the three main flavors — converging, same-direction, and fast-slow — dozens of problems reduce to fill-in-the-blank.

1. The Three Flavors of Two Pointers

Two pointers isn't one technique — it's three closely related ones. Each has its own template and its own class of problems.

Flavor A: Converging (Opposite Ends)

```
arr = [1, 3, 5, 7, 9, 11]
      ↑       ↑
    left    right
```

left starts at 0, right at $n-1$.
They move TOWARD each other until they meet.

Classic uses: reverse, palindrome, 2-Sum on sorted array,
Container With Most Water, 3-Sum.

Flavor B: Same Direction (Fast & Slow)

```
arr = [0, 0, 1, 0, 2, 3]
      ↑       ↑
    slow    fast
```

Both start at 0. Fast scans ahead;
slow advances only under a condition.

Classic uses: Move Zeroes, Remove Duplicates,
Partition Array, Remove Element.

Flavor C: Fast-Slow (Different Speeds)

```
Linked list: 1 → 2 → 3 → 4 → 5
```

slow moves 1 step at a time.
fast moves 2 steps at a time.

If there's a cycle, fast catches up to slow.
If there's no cycle, fast reaches null first.

Classic uses: Linked List Cycle, Middle of List,
Happy Number, palindrome linked list.

Same idea, different speeds: All three flavors keep total work $O(n)$ because each pointer moves at most n steps. What changes is HOW the pointers move: toward each other, both forward, or at different speeds. Learn to spot which flavor applies from the problem shape.

2. When to Reach for Two Pointers

Signal words that scream "two pointers" — especially the converging kind:

- The array is SORTED, and you need to find a pair matching a condition
- Compare/process elements from BOTH ends toward the middle
- "Find a pair/triplet/quadruplet with sum = target"
- Palindrome check on an array or string
- Partition the array in-place (move certain values to one side)
- Cycle detection or middle-finding on a linked list
- Container / capacity problems with two boundaries

Two pointers vs sliding window: Both use two indices. The key difference: sliding window is about a CONTIGUOUS RANGE of elements — you track state for the range and shrink/expand it. Two pointers just uses two INDICES independently — you don't necessarily care about what's between them. Same tool family, different mental model.

3. Converging Pointer Template

```
int left = 0, right = arr.length - 1;

while (left < right) {
    // Look at arr[left] and arr[right]
    if (satisfies_condition) {
        // record answer / do work
        left++;
        right--;
    } else if (needs_bigger_value) {
        left++; // move to a larger value (assuming sorted)
    } else {
        right--; // move to a smaller value
    }
}
```

```
}  
}
```

Two decision knobs: (1) how do you compare left and right? and (2) which side moves when the condition doesn't match? The rest is problem-specific.

4. Classic Problem 1 — Two Sum on a Sorted Array

Given a SORTED array and target, find two indices whose values sum to the target. $O(n)$ with two pointers — no HashMap needed since the array is sorted.

```
public int[] twoSumSorted(int[] arr, int target) {  
    int left = 0, right = arr.length - 1;  
    while (left < right) {  
        int sum = arr[left] + arr[right];  
        if (sum == target) return new int[]{left, right};  
        else if (sum < target) left++;        // need a bigger value  
        else right--;                        // need a smaller value  
    }  
    return new int[]{-1, -1};  
}  
  
// Time:  $O(n)$     Space:  $O(1)$ 
```

Why This Works

Because the array is sorted, if $\text{arr}[\text{left}] + \text{arr}[\text{right}]$ is too small, no smaller value on the right side will help — you must move LEFT forward. Symmetric logic when too big. Each pointer only moves in one direction, so total work is bounded by n steps.

For UNSORTED arrays: Sort first ($O(n \log n)$) and lose the original indices, or use a HashMap (Ch 20). Two pointers requires the sortedness — that's what lets you skip half the search space each step.

5. Classic Problem 2 — 3-Sum

Given an array, find ALL unique triplets (a, b, c) that sum to zero. This is one of the most famous coding interview problems — the standard solution combines SORTING with two-pointer sweeping.

Strategy

1. Sort the array
2. For each index i , use two pointers to find pairs summing to $-\text{arr}[i]$
3. SKIP duplicates at each level to avoid duplicate triplets

```
public List<List<Integer>> threeSum(int[] nums) {  
    Arrays.sort(nums);  
    List<List<Integer>> result = new ArrayList<>();  
  
    for (int i = 0; i < nums.length - 2; i++) {
```

```

        // Skip duplicate first elements
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1, right = nums.length - 1;
        int target = -nums[i];

        while (left < right) {
            int sum = nums[left] + nums[right];
            if (sum == target) {
                result.add(Arrays.asList(nums[i], nums[left],
nums[right]));

                // Skip duplicates on both sides
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1])
right--;

                left++;
                right--;
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
    }
    return result;
}

// Time: O(n^2)    Space: O(1) extra (ignoring output)

```

Dry Run for nums = [-1, 0, 1, 2, -1, -4]

```

Sorted:  [-4, -1, -1, 0, 1, 2]

i=0 (-4), target = 4:
    left=1, right=5: -1 + 2 = 1 < 4 → left++
    left=2, right=5: -1 + 2 = 1 < 4 → left++
    left=3, right=5: 0 + 2 = 2 < 4 → left++
    left=4, right=5: 1 + 2 = 3 < 4 → left++, now left==right, stop

i=1 (-1), target = 1:
    left=2, right=5: -1 + 2 = 1 ✓ → add [-1, -1, 2]
                        skip duplicate -1: left=3
                        left++, right-- → left=3, right=4
    left=3, right=4: 0 + 1 = 1 ✓ → add [-1, 0, 1]
                        left++, right-- → left=4, right=3, stop

i=2 (-1): duplicate of i=1 → skip

```

```
i=3 (0), target = 0:
  left=4, right=5: 1 + 2 = 3 > 0 → right--, now left==right, stop

Result: [[-1, -1, 2], [-1, 0, 1]]
```

The duplicate-skipping is the tricky part: There are THREE places to skip duplicates in 3-Sum: the outer i-loop, and BOTH inner pointers after a match. Miss any of them and you'll get duplicate triplets in the result. Always test with input containing repeated values.

6. Classic Problem 3 — Container With Most Water

Given an array of heights, find the two lines that together with the x-axis form a container holding the MOST water. $\text{Area} = \min(\text{height}[\text{left}], \text{height}[\text{right}]) \times (\text{right} - \text{left})$.

```
heights = [1, 8, 6, 2, 5, 4, 8, 3, 7]
```

```
If we pick indices 1 and 8:
  width  = 8 - 1 = 7
  height = min(8, 7) = 7
  area   = 7 × 7 = 49 ← the maximum
```

```
Strategy: start with the widest possible container (left=0, right=n-1)
1)          and try to improve by moving the SHORTER wall inward.
```

The Insight

At every step, the width DECREASES. So to have any hope of a bigger area, the height must INCREASE. Moving the taller wall inward can only reduce the height (the shorter wall still limits it). Only moving the SHORTER wall gives a chance at finding something taller.

```
public int maxArea(int[] heights) {
    int left = 0, right = heights.length - 1;
    int best = 0;

    while (left < right) {
        int h = Math.min(heights[left], heights[right]);
        best = Math.max(best, h * (right - left));

        if (heights[left] < heights[right]) left++;
        else right--;
    }
    return best;
}

// Time: O(n)   Space: O(1)
```

7. Classic Problem 4 — Move Zeroes (Same-Direction Pointers)

Given an array, move all 0s to the end while keeping the RELATIVE ORDER of non-zero elements. Must be done in-place, no extra array.

```
public void moveZeroes(int[] nums) {
    int slow = 0;

    // slow marks where the next non-zero should go
    for (int fast = 0; fast < nums.length; fast++) {
        if (nums[fast] != 0) {
            int tmp = nums[slow];
            nums[slow] = nums[fast];
            nums[fast] = tmp;
            slow++;
        }
    }
}
```

Dry Run for [0, 1, 0, 3, 12]

```
Start: [0, 1, 0, 3, 12]  slow=0, fast=0

fast=0: nums[0]=0, skip
fast=1: nums[1]=1, swap slow(0) and fast(1)
        [1, 0, 0, 3, 12]  slow=1
fast=2: nums[2]=0, skip
fast=3: nums[3]=3, swap slow(1) and fast(3)
        [1, 3, 0, 0, 12]  slow=2
fast=4: nums[4]=12, swap slow(2) and fast(4)
        [1, 3, 12, 0, 0]  slow=3

Result: [1, 3, 12, 0, 0] ✓
```

The pattern: slow = "where the next valid element goes." fast = "scanner looking for valid elements."
This same shape works for Remove Element, Remove Duplicates from Sorted Array, Partition Array, and many other in-place restructuring problems.

8. Classic Problem 5 — Remove Duplicates from Sorted Array

Given a SORTED array, remove duplicates IN-PLACE such that each element appears only once. Return the new length.

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;

    int slow = 0; // last position with a unique value
    for (int fast = 1; fast < nums.length; fast++) {
```

```

        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;
}

// Example: [1, 1, 2, 3, 3, 4]
// After:   [1, 2, 3, 4, ?, ?]    return 4

```

Same slow-fast pattern. slow points at the last confirmed unique value; fast scans ahead looking for new ones.

9. Classic Problem 6 — Valid Palindrome

Check if a string is a palindrome, ignoring non-alphanumeric characters and letter case.

```

public boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;

    while (left < right) {
        // Skip non-alphanumeric on the left
        while (left < right && !Character.isLetterOrDigit(s.charAt(left)))
            left++;
        // Skip non-alphanumeric on the right
        while (left < right && !
            Character.isLetterOrDigit(s.charAt(right))) right--;

        if (Character.toLowerCase(s.charAt(left))
            != Character.toLowerCase(s.charAt(right))) return false;

        left++;
        right--;
    }
    return true;
}

// "A man, a plan, a canal: Panama" → true

```

Classic converging pointer with inner skip loops for irrelevant characters. Each character is visited at most twice — still $O(n)$.

10. Classic Problem 7 — Linked List Cycle (Fast-Slow Pointer)

Given a linked list, determine if it has a cycle. Floyd's Tortoise and Hare algorithm: fast pointer moves 2 steps, slow moves 1. If there's a cycle, they meet. If not, fast hits null.

```

public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;

```

```

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

// Time: O(n)    Space: O(1)  – no HashSet needed!

```

Why This Works

If there's a cycle, once both pointers enter it, the fast pointer catches up to slow at rate 1 per step (fast gains 1 step per iteration). Eventually they land on the same node. If there's no cycle, fast reaches null first — bounded loop, guaranteed termination.

Same pattern, related problems: The fast-slow variant also finds the MIDDLE of a linked list (when fast hits the end, slow is at the middle), detects the CYCLE START (Floyd's second pass), determines if a linked list is a PALINDROME, and computes if a number is HAPPY. Same trick, different data.

11. Classic Problem 8 — Trapping Rain Water (Two Pointer Alternative)

You solved this with a monotonic stack in Chapter 39. There's an $O(n)$ two-pointer version that uses $O(1)$ extra space — often preferred in interviews.

```

public int trap(int[] heights) {
    int left = 0, right = heights.length - 1;
    int leftMax = 0, rightMax = 0;
    int water = 0;

    while (left < right) {
        if (heights[left] < heights[right]) {
            if (heights[left] >= leftMax) leftMax = heights[left];
            else water += leftMax - heights[left];
            left++;
        } else {
            if (heights[right] >= rightMax) rightMax = heights[right];
            else water += rightMax - heights[right];
            right--;
        }
    }
    return water;
}

// Time: O(n)    Space: O(1)

```


Intuition: water above a bar is limited by the SMALLER of the tallest bars on either side. Whichever side is shorter, we know the answer for THAT side is fully determined by its own max — safe to compute and move that pointer.

12. Classic Problem 9 — Sort Colors (Dutch National Flag)

Given an array containing only 0s, 1s, and 2s, sort it in-place in one pass. This is the DUTCH NATIONAL FLAG problem — solved with THREE pointers (a specialized two-pointer variant).

```
public void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;

    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums, low, mid);
            low++;
            mid++;
        } else if (nums[mid] == 1) {
            mid++;
        } else { // nums[mid] == 2
            swap(nums, mid, high);
            high--;
            // note: don't advance mid — new value at mid needs inspection
        }
    }
}

void swap(int[] a, int i, int j) {
    int tmp = a[i]; a[i] = a[j]; a[j] = tmp;
}
```

Three regions: [0..low) holds 0s, [low..mid) holds 1s, (high..end] holds 2s. mid is the exploration pointer. This is basically two-pointers upgraded to partition into three groups.

13. Common Two-Pointer Problem Patterns

Problem shape	Approach
Find pair with sum = target (sorted)	Converging: left++ if sum too small, right-- otherwise
Find triplet with sum = target	Sort + outer loop + converging inner
Palindrome check	Converging + skip non-alphanumeric if needed
Reverse array / string in-place	Converging + swap
Container with most water	Converging + move shorter wall inward
Remove element / duplicates in-place	Same-direction: slow marks position, fast scans

Problem shape	Approach
Move certain values to end/start	Same-direction with slow marking the boundary
Partition into 3 groups (Dutch flag)	Three pointers: low, mid, high
Cycle detection (linked list)	Fast-slow (Floyd's)
Find middle of a linked list	Fast-slow: fast reaches end when slow at middle
Merge two sorted arrays	Two pointers walking each array

14. Complexity Analysis

Problem	Time	Space
Two Sum (sorted array)	$O(n)$	$O(1)$
3-Sum	$O(n^2)$	$O(1)$ extra
4-Sum	$O(n^3)$	$O(1)$ extra
Container With Most Water	$O(n)$	$O(1)$
Move Zeroes / Remove Element	$O(n)$	$O(1)$
Valid Palindrome	$O(n)$	$O(1)$
Linked List Cycle	$O(n)$	$O(1)$
Trapping Rain Water	$O(n)$	$O(1)$
Sort Colors (Dutch Flag)	$O(n)$	$O(1)$

Two pointers is a space-optimizer as much as a time-optimizer. Many problems solvable with HashMap in $O(n)$ time / $O(n)$ space can be done with two pointers in $O(n)$ time / $O(1)$ space — often preferable for large inputs.

15. Common Mistakes

Mistake 1: Using two pointers on unsorted array without sorting first

```
int[] nums = {3, 1, 5, 2, 4};
// Trying two-pointer 2-Sum on this UNSORTED array will miss valid pairs.
// Sort first, or use HashMap approach.
```

Mistake 2: Forgetting to skip duplicates in 3-Sum

There are THREE places to skip: outer loop, left pointer after match, right pointer after match. Miss any and you get duplicate triplets. Always test with inputs containing repeated values.

Mistake 3: Advancing mid in Dutch Flag when swapping with high

```
if (nums[mid] == 2) {
    swap(nums, mid, high);
    high--;
    mid++;    // x WRONG – the swapped-in value at mid isn't inspected
}

// Correct: DON'T advance mid – the new value could be 0, 1, or 2
```

Mistake 4: Infinite loop when pointers don't move

Every iteration MUST make progress — advance at least one pointer. Otherwise the loop hangs. Common trap in palindrome-with-skip: forgetting to advance after checking equality.

Mistake 5: Wrong loop condition (< vs ≤)

```
while (left < right)    // ✓ for pair problems – pointers must be distinct
while (left <= right)   // ✓ for single-element problems (e.g., binary
search)

// Mixing these up produces off-by-one bugs.
```

Mistake 6: Not handling edge cases (empty, single element)

```
// Move Zeroes on []? On [5]? On [0]?
// Always test these – two pointer solutions usually handle them cleanly
// with correct loop conditions, but assumptions can break for length ≤ 1.
```

Mistake 7: Using fast.next != null instead of fast != null && fast.next != null

```
while (fast.next != null) { ... }    // x NullPointerException
if fast becomes null
while (fast != null && fast.next != null) { }    // ✓ correct
// Because fast moves 2 steps at a time, both fast and fast.next need
checking.
```

16. Best Practices

- Identify the FLAVOR first — converging, same-direction, or fast-slow
- For sum-related problems, sort first if the array isn't sorted
- Always advance at least one pointer per iteration to guarantee termination
- Use while (left < right) for pair problems; watch for the boundary carefully
- For 3-Sum and 4-Sum, skip duplicates at EVERY LEVEL

- For fast-slow on linked lists, always check both `fast != null` AND `fast.next != null`
- Two pointers usually gives $O(1)$ space — a common tie-breaker over HashMap solutions
- Draw the pointer positions on paper for a small input before coding

17. Quick Reference

Concept	Meaning
Two Pointers	Two indices moving intelligently through data
Converging	Left and right start at ends, move toward each other
Same-direction	Slow marks position; fast scans ahead
Fast-slow	Different speeds — for cycles, middle-finding
Floyd's algorithm	Fast-slow for cycle detection
Dutch National Flag	Three pointers to partition into three groups
Loop condition	<code>while (left < right)</code> for pair problems
Sort first	Often required for sum-based two-pointer solutions
Skip duplicates	In outer loop AND both inner pointers for 3-Sum
Space	Usually $O(1)$ — a key advantage over HashMap

18. Practice Problems

Try in order — earlier ones warm up patterns that harder ones build on.

Converging Warm-ups

- Two Sum II — Input Array is Sorted (LeetCode 167).
- Reverse String (LeetCode 344).
- Valid Palindrome (LeetCode 125).
- Valid Palindrome II — with at most one deletion (LeetCode 680).
- Squares of a Sorted Array (LeetCode 977).

Multi-Sum Family

- 3Sum (LeetCode 15).
- 3Sum Closest — find triplet with sum closest to target (LeetCode 16).
- 3Sum Smaller — count triplets with sum < target (LeetCode 259).

12. 4Sum (LeetCode 18).

Container / Area

13. Container With Most Water (LeetCode 11).

14. Trapping Rain Water (LeetCode 42) — two-pointer variant.

Same-Direction (Slow/Fast)

15. Move Zeroes (LeetCode 283).

16. Remove Duplicates from Sorted Array (LeetCode 26).

17. Remove Duplicates from Sorted Array II — allow up to 2 (LeetCode 80).

18. Remove Element (LeetCode 27).

19. Sort Colors — Dutch National Flag (LeetCode 75).

20. Partition Labels (LeetCode 763).

Fast-Slow (Linked Lists)

21. Linked List Cycle (LeetCode 141).

22. Linked List Cycle II — find the cycle start (LeetCode 142).

23. Middle of the Linked List (LeetCode 876).

24. Happy Number (LeetCode 202) — same trick, on integers.

25. Palindrome Linked List (LeetCode 234).

Merge & String

26. Merge Sorted Array (LeetCode 88) — merge from the end.

27. Backspace String Compare (LeetCode 844).

28. Is Subsequence (LeetCode 392).

19. Final Takeaway

Two pointers is the most reusable technique in DSA. You've been using it since Chapter 8's pattern-printing days — this chapter just formalizes the three flavors and shows how far they scale. Once you spot which flavor a problem needs, the code writes itself.

For interviews, master five problems: Two Sum sorted, 3-Sum, Container With Most Water, Move Zeroes, and Linked List Cycle. Those five hit every flavor and prepare you for 80% of two-pointer questions you'll ever face. The rest is just recognizing the pattern from the problem shape.

Key Takeaway: Two pointers = two indices moving intelligently through data. Three flavors: CONVERGING (opposite ends, for sums/palindromes), SAME-DIRECTION (slow-fast for in-place restructuring), and FAST-SLOW (different speeds, for linked-list cycles). Total work is $O(n)$ because

each pointer moves at most n steps. Sort first for sum-based problems. Skip duplicates in 3-Sum. It's a space-optimizer — often gives $O(1)$ space where HashMap needs $O(n)$.

20. What's Next?

With two pointers covered in depth, the remaining specialized DSA topics are:

- Greedy Algorithms — activity selection, interval scheduling, Huffman coding
- Segment Tree / Fenwick Tree — range queries WITH updates
- Advanced Strings — KMP, Rabin-Karp, Z-algorithm
- Advanced Graphs — Bellman-Ford, Floyd-Warshall, articulation points, bridges
- Number Theory — GCD tricks, primes, modular arithmetic, combinatorics
- Binary Search patterns in depth — search on answer space, rotated arrays
- System Design — the natural next step beyond DSA for interviews